Problem statement: AI-powered resume screening tools are increasingly used by companies to filter job applicants. However, these systems are trained on historical hiring data that contains racial and gender biases. This project investigates whether existing AI models (such as ChatGPT, Gemini, and Claude) exhibit bias when screening resumes that are identical in qualifications but differ only in race, gender, marital status, or country of origin.

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving adult.csv to adult.csv
```

```
#data check
import pandas as pd

df = pd.read_csv('adult.csv')
print(df.shape)
print(df.head())
```

```
(48842, 15)
   age  workclass  fnlwgt     education  educational-num      marital-status  \
0   25    Private  226802          11th                7       Never-married
1   38    Private   89814       HS-grad                9  Married-civ-spouse
2   28  Local-gov  336951    Assoc-acdm               12  Married-civ-spouse
3   44    Private  160323  Some-college               10  Married-civ-spouse
4   18          ?  103497  Some-college               10       Never-married

          occupation relationship   race  gender  capital-gain  capital-loss  \
0  Machine-op-inspct    Own-child  Black    Male             0             0
1    Farming-fishing      Husband  White    Male             0             0
2    Protective-serv      Husband  White    Male             0             0
3  Machine-op-inspct      Husband  Black    Male          7688             0
4                  ?    Own-child  White  Female             0             0

   hours-per-week native-country income
0              40  United-States  <=50K
1              50  United-States  <=50K
2              40  United-States   >50K
3              40  United-States   >50K
4              30  United-States  <=50K
```

```
#data preprocessing
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('adult.csv')
df.replace('?', np.nan, inplace=True)
df.dropna(inplace=True)

df = df[['age', 'race', 'gender', 'education', 'marital-status', 'occupation', 'relationship', 'native-country']]

df = df.rename(columns={
```

```
    'marital-status': 'marital_status',
    'native-country': 'country'
})

df['gender'] = df['gender'].map({'Male': 1, 'Female': 0})

race_dummies = pd.get_dummies(df['race'], prefix='race')
df = pd.concat([df, race_dummies], axis=1)
df = df.drop('race', axis=1)

marital_dummies = pd.get_dummies(df['marital_status'], prefix='marital')
df = pd.concat([df, marital_dummies], axis=1)
df = df.drop('marital_status', axis=1)

relationship_dummies = pd.get_dummies(df['relationship'], prefix='relationship')
df = pd.concat([df, relationship_dummies], axis=1)
df = df.drop('relationship', axis=1)

education_dummies = pd.get_dummies(df['education'], prefix='education')
df = pd.concat([df, education_dummies], axis=1)
df = df.drop('education', axis=1)

occupation_dummies = pd.get_dummies(df['occupation'], prefix='occupation')
df = pd.concat([df, occupation_dummies], axis=1)
df = df.drop('occupation', axis=1)

country_dummies = pd.get_dummies(df['country'], prefix='country')
df = pd.concat([df, country_dummies], axis=1)
df = df.drop('country', axis=1)

scaler = StandardScaler()
df['age'] = scaler.fit_transform(df[['age']])

df.to_csv('adult_preprocessed.csv', index=False)

print(f"Preprocessed {len(df)} records")
print(f"Total columns: {len(df.columns)}")
print(df.head())
```

```
Preprocessed 45222 records
Total columns: 91
        age  gender  race_Amer-Indian-Eskimo  race_Asian-Pac-Islander  \
0 -1.024983       1                    False                    False
1 -0.041455       1                    False                    False
2 -0.798015       1                    False                    False
3  0.412481       1                    False                    False
5 -0.344079       1                    False                    False

   race_Black  race_Other  race_White  marital_Divorced  \
0        True       False       False             False
1       False       False        True             False
2       False       False        True             False
3        True       False       False             False
5       False       False        True             False

   marital_Married-AF-spouse  marital_Married-civ-spouse  ...  \
0                      False                       False  ...
```

```
1                    False                       True  ...
2                    False                       True  ...
3                    False                       True  ...
5                    False                      False  ...

   country_Portugal  country_Puerto-Rico  country_Scotland  country_South  \
0             False                False             False          False
1             False                False             False          False
2             False                False             False          False
3             False                False             False          False
5             False                False             False          False

   country_Taiwan  country_Thailand  country_Trinadad&Tobago  \
0           False             False                    False
1           False             False                    False
2           False             False                    False
3           False             False                    False
5           False             False                    False

   country_United-States  country_Vietnam  country_Yugoslavia
0                   True            False               False
1                   True            False               False
2                   True            False               False
3                   True            False               False
5                   True            False               False

[5 rows x 91 columns]
```

```
#descriptive statistics
import pandas as pd

df = pd.read_csv('adult_preprocessed.csv')

print("=" * 70)
print("DESCRIPTIVE STATISTICS")
print("=" * 70)

print("\nAGE STATISTICS:")
print(f"Mean: {df['age'].mean():.2f}")
print(f"Median: {df['age'].median():.2f}")
print(f"Mode: {df['age'].mode()[0]:.2f}")
print(f"Variance: {df['age'].var():.2f}")
print(f"Standard Deviation: {df['age'].std():.2f}")
print(f"IQR: {df['age'].quantile(0.75) - df['age'].quantile(0.25):.2f}")
print(f"Skewness: {df['age'].skew():.2f}")
print(f"Kurtosis: {df['age'].kurtosis():.2f}")

print("\nGENDER DISTRIBUTION:")
print(df['gender'].value_counts())
print(f"Proportion Male: {df['gender'].mean():.2%}")

race_cols = [c for c in df.columns if c.startswith('race_')]
print("\nRACE DISTRIBUTION:")
for col in race_cols:
    print(f"{col}: {df[col].sum()} ({df[col].mean():.2%})")
```

```
======================================================================
DESCRIPTIVE STATISTICS
```

```
======================================================================

AGE STATISTICS:
Mean: -0.00
Median: -0.12
Mode: -0.19
Variance: 1.00
Standard Deviation: 1.00
IQR: 1.44
Skewness: 0.53
Kurtosis: -0.16

GENDER DISTRIBUTION:
gender
1    30527
0    14695
Name: count, dtype: int64
Proportion Male: 67.50%

RACE DISTRIBUTION:
race_Amer-Indian-Eskimo: 435 (0.96%)
race_Asian-Pac-Islander: 1303 (2.88%)
race_Black: 4228 (9.35%)
race_Other: 353 (0.78%)
race_White: 38903 (86.03%)
```

```
#univariate visualizations
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('adult_preprocessed.csv')

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

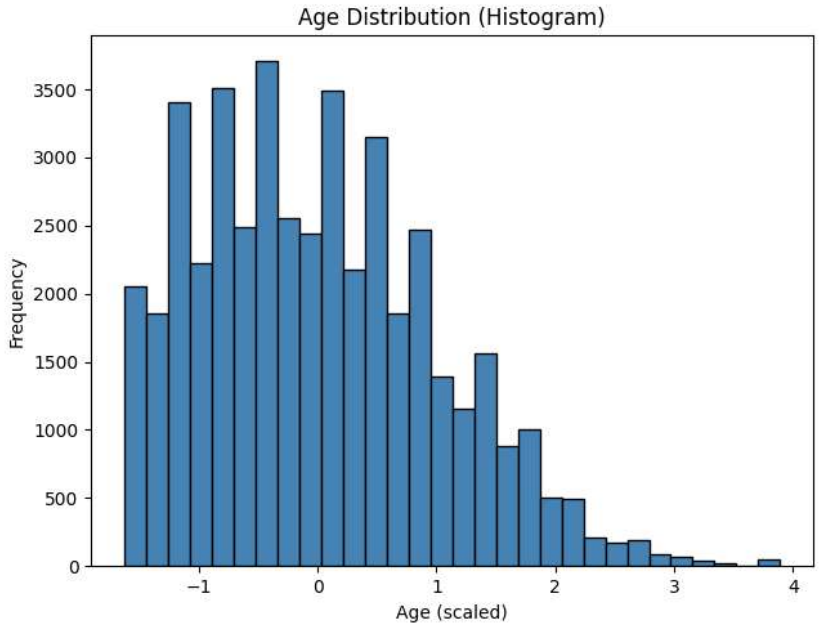
axes[0, 0].hist(df['age'], bins=30, color='steelblue', edgecolor='black')
axes[0, 0].set_title('Age Distribution (Histogram)')
axes[0, 0].set_xlabel('Age (scaled)')
axes[0, 0].set_ylabel('Frequency')

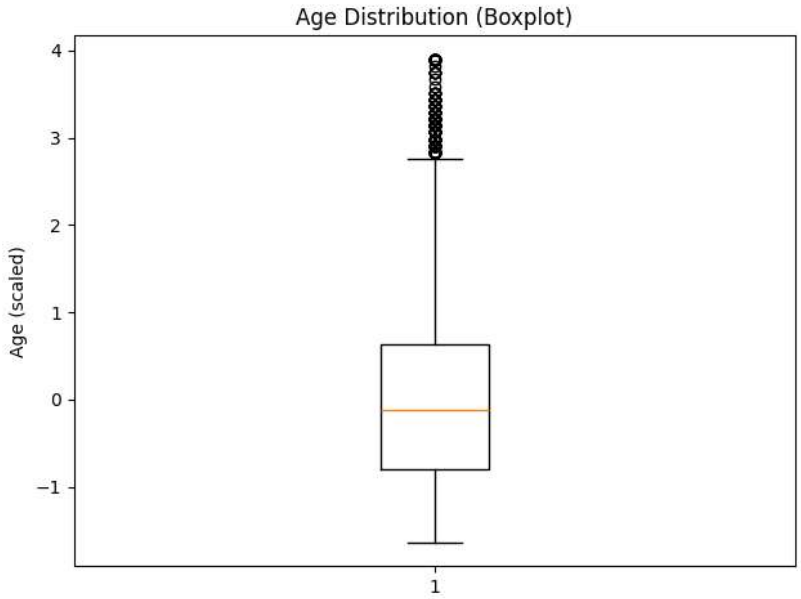
axes[0, 1].boxplot(df['age'])
axes[0, 1].set_title('Age Distribution (Boxplot)')
axes[0, 1].set_ylabel('Age (scaled)')

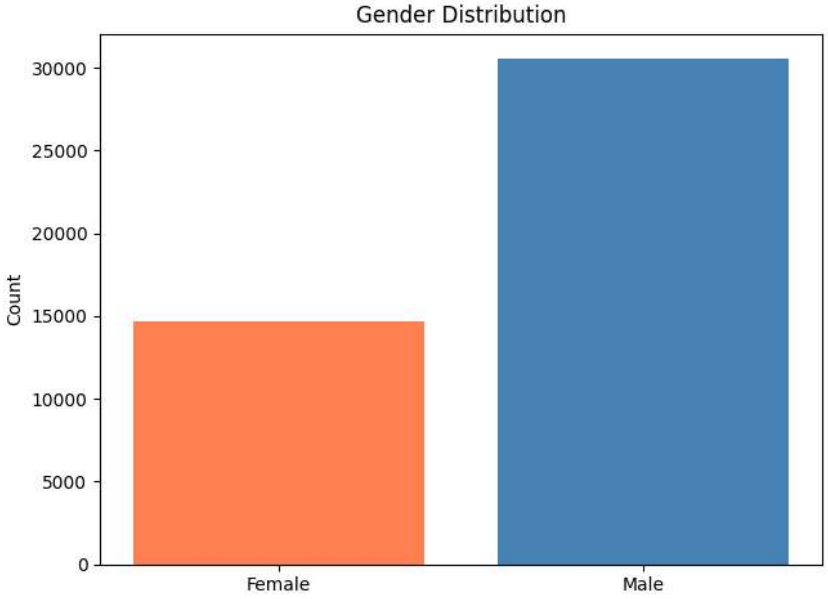
gender_counts = df['gender'].value_counts()
axes[1, 0].bar(['Female', 'Male'], [gender_counts.get(0, 0), gender_counts.get(1, 0)], color=['coral', 'steelblue'])
axes[1, 0].set_title('Gender Distribution')
axes[1, 0].set_ylabel('Count')

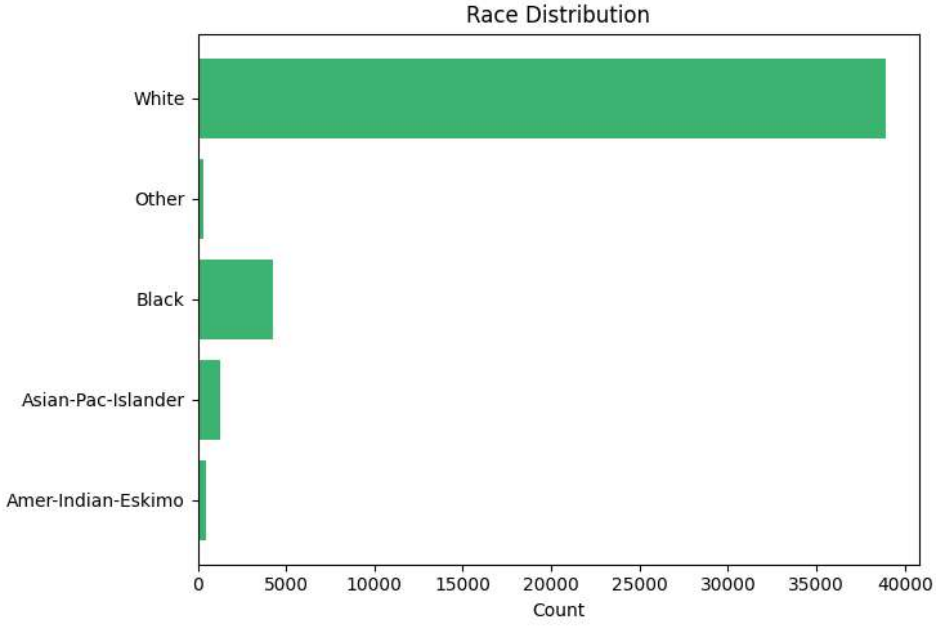
race_cols = [c for c in df.columns if c.startswith('race_')]
race_counts = [df[c].sum() for c in race_cols]
race_labels = [c.replace('race_', '') for c in race_cols]
axes[1, 1].barh(race_labels, race_counts, color='mediumseagreen')
axes[1, 1].set_title('Race Distribution')
axes[1, 1].set_xlabel('Count')

plt.tight_layout()
plt.show()
```










```
#outlier detection
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from scipy import stats

df = pd.read_csv('adult_preprocessed.csv')

Q1 = df['age'].quantile(0.25)
Q3 = df['age'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers_iqr = df[(df['age'] < lower) | (df['age'] > upper)]
z_scores = np.abs(stats.zscore(df['age']))
outliers_z = df[z_scores > 3]

print("=" * 70)
print("OUTLIER DETECTION")
print("=" * 70)
print(f"\nIQR Method:")
print(f"Q1: {Q1:.2f}, Q3: {Q3:.2f}, IQR: {IQR:.2f}")
print(f"Lower bound: {lower:.2f}, Upper bound: {upper:.2f}")
print(f"Outliers found: {len(outliers_iqr)}")
print(f"\nZ-score Method:")
print(f"Outliers found: {len(outliers_z)}")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].boxplot(df['age'])
axes[0].set_title('Age Boxplot (IQR Method)')
axes[0].set_ylabel('Age (scaled)')

axes[1].scatter(range(len(df)), df['age'], alpha=0.3, s=1)
axes[1].axhline(y=upper, color='r', linestyle='--', label='Upper bound')
axes[1].axhline(y=lower, color='r', linestyle='--', label='Lower bound')
axes[1].set_title('Age Scatter with Outlier Bounds')
axes[1].set_ylabel('Age (scaled)')
axes[1].legend()

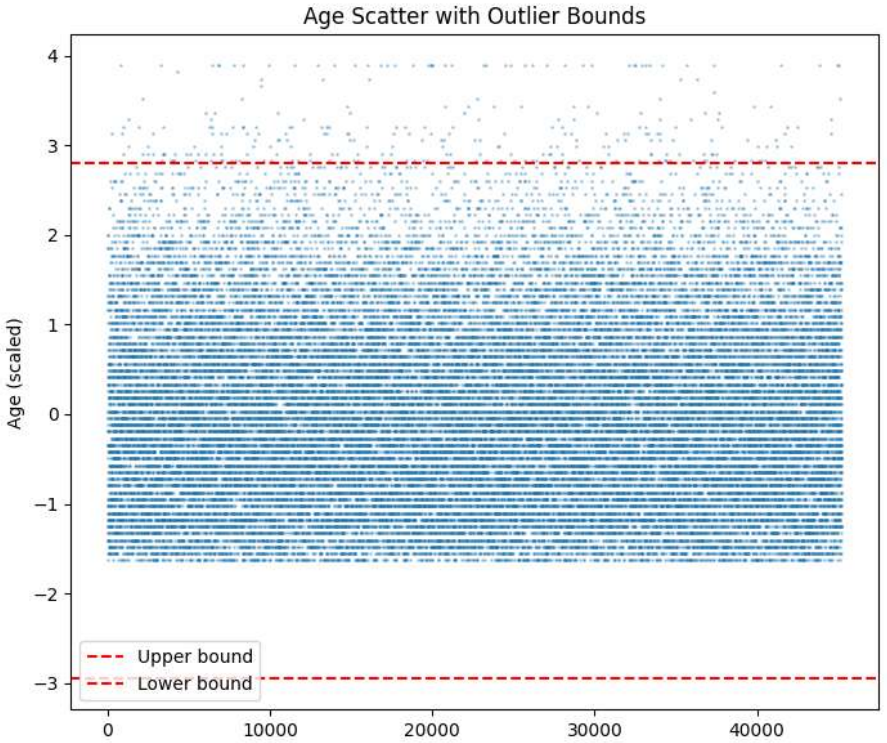
plt.tight_layout()
plt.show()
```

```
======================================================================
OUTLIER DETECTION
======================================================================

IQR Method:
Q1: -0.80, Q3: 0.64, IQR: 1.44
Lower bound: -2.95, Upper bound: 2.80
Outliers found: 269

Z-score Method:
Outliers found: 163
```






```
#bivariate visualization
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('adult_fixed.csv')

# Clean race labels for plotting
df['race_label'] = df['race'].str.replace('-', ' ')

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Education Level Distribution by Race (Bar Chart)
# Create a crosstab of education vs race
```

```
edu_race = pd.crosstab(df['race_label'], df['education'])
# Normalize to show percentage within each race
edu_race_pct = edu_race.div(edu_race.sum(axis=1), axis=0) * 100
# Plot a stacked bar or simple bar for a few key education levels
key_edu = ['HS-grad', 'Bachelors', 'Masters', 'Doctorate']
edu_race_pct[key_edu].plot(kind='bar', ax=axes[0], colormap='viridis')
axes[0].set_title('Education Distribution by Race (% within Race)', fontsize=14)
axes[0].set_ylabel('Percentage')
axes[0].legend(title='Education', bbox_to_anchor=(1.05, 1), loc='upper left')

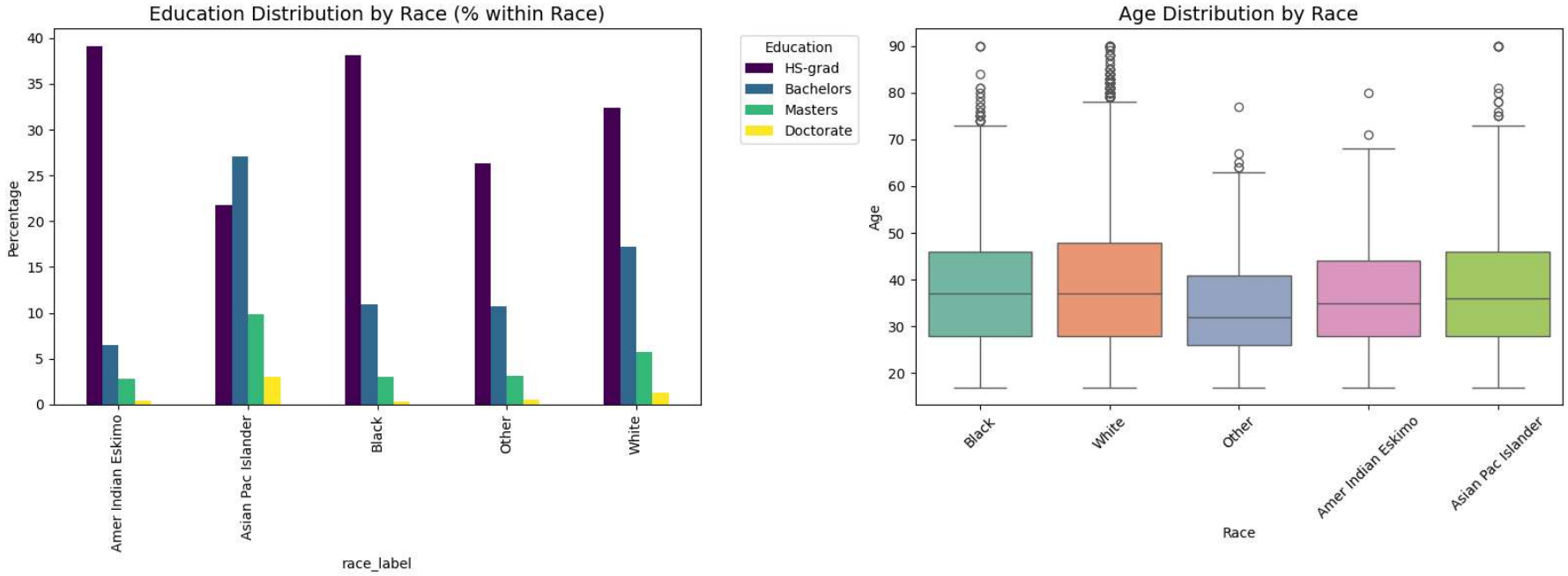
# Plot 2: Age Distribution by Race (Box Plot)
sns.boxplot(x='race_label', y='age', data=df, ax=axes[1], palette='Set2')
axes[1].set_title('Age Distribution by Race', fontsize=14)
axes[1].set_xlabel('Race')
axes[1].set_ylabel('Age')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_1917/2305968028.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='race_label', y='age', data=df, ax=axes[1], palette='Set2')
```




```
#multivariate visualizations
import pandas as pd
import matplotlib.pyplot as plt
```

```
from pandas.plotting import parallel_coordinates, andrews_curves
import seaborn as sns

df = pd.read_csv('adult_preprocessed.csv')

race_cols = [c for c in df.columns if c.startswith('race_')]
df['race_label'] = df[race_cols].idxmax(axis=1).str.replace('race_', '')

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

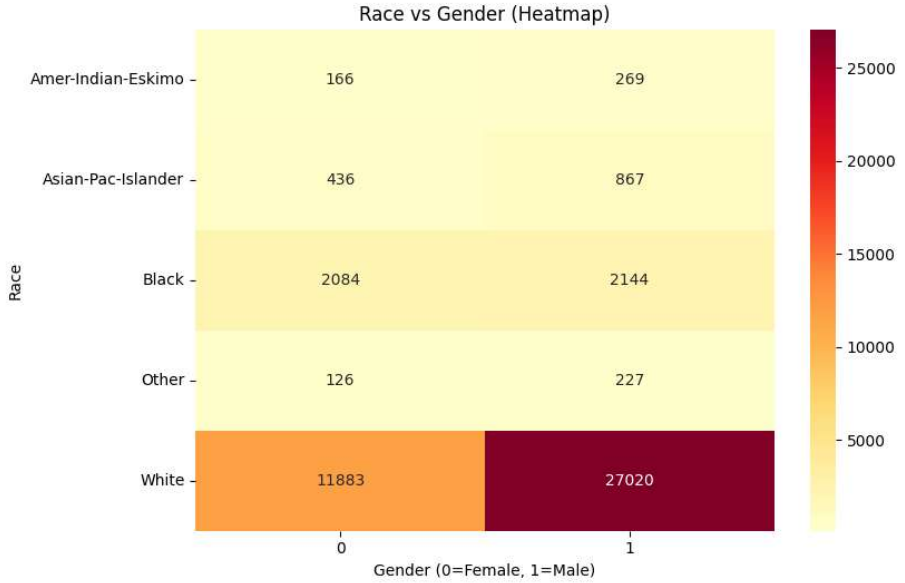
pivot = pd.crosstab(df['race_label'], df['gender'])
sns.heatmap(pivot, annot=True, fmt='d', cmap='YlOrRd', ax=axes[0, 0])
axes[0, 0].set_title('Race vs Gender (Heatmap)')
axes[0, 0].set_ylabel('Race')
axes[0, 0].set_xlabel('Gender (0=Female, 1=Male)')

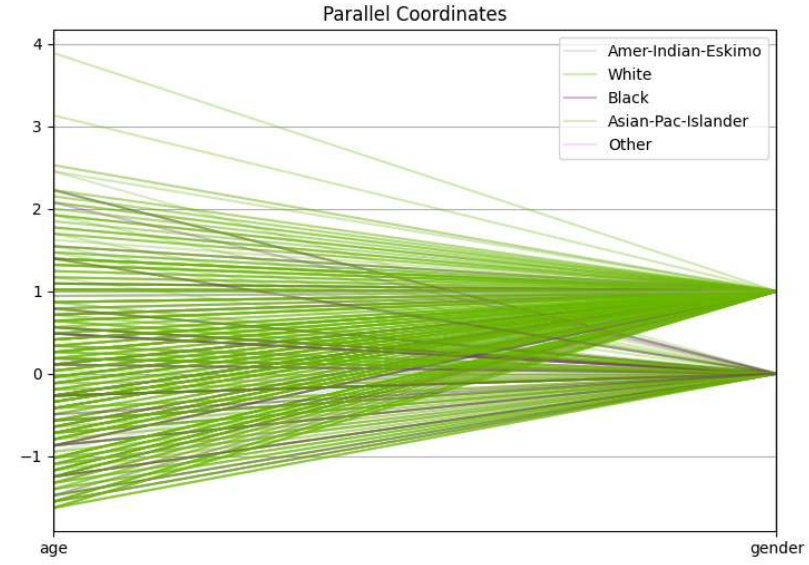
sample = df.sample(500, random_state=42)
cols_to_plot = ['age', 'gender', 'race_label']
parallel_coordinates(sample[cols_to_plot], 'race_label', ax=axes[0, 1], alpha=0.3)
axes[0, 1].set_title('Parallel Coordinates')

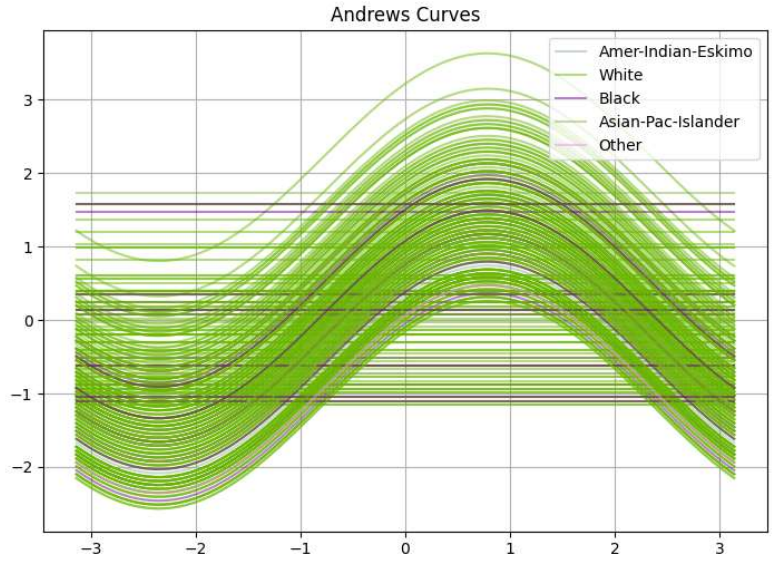
sample2 = df.sample(300, random_state=42)
andrews_curves(sample2[['age', 'gender', 'race_label']], 'race_label', ax=axes[1, 0], alpha=0.5)
axes[1, 0].set_title('Andrews Curves')

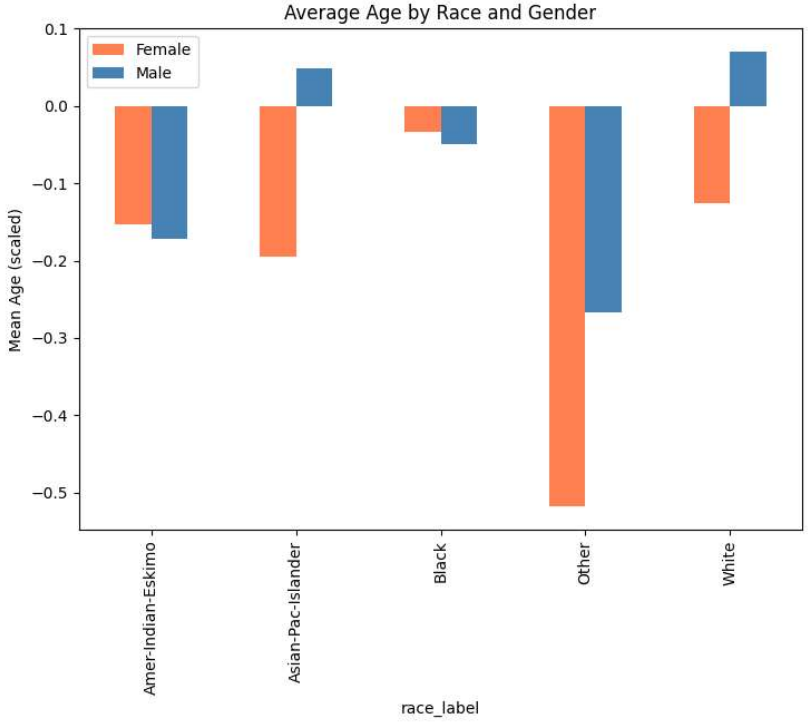
avg_age = df.groupby(['race_label', 'gender'])['age'].mean().unstack()
avg_age.plot(kind='bar', ax=axes[1, 1], color=['coral', 'steelblue'])
axes[1, 1].set_title('Average Age by Race and Gender')
axes[1, 1].set_ylabel('Mean Age (scaled)')
axes[1, 1].legend(['Female', 'Male'])

plt.tight_layout()
plt.show()
```










```
#pca and t-sne
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

df = pd.read_csv('adult_preprocessed.csv')

race_cols = [c for c in df.columns if c.startswith('race_')]
df['race_label'] = df[race_cols].idxmax(axis=1).str.replace('race_', '')

features = ['age', 'gender'] + race_cols
X = df[features].values

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

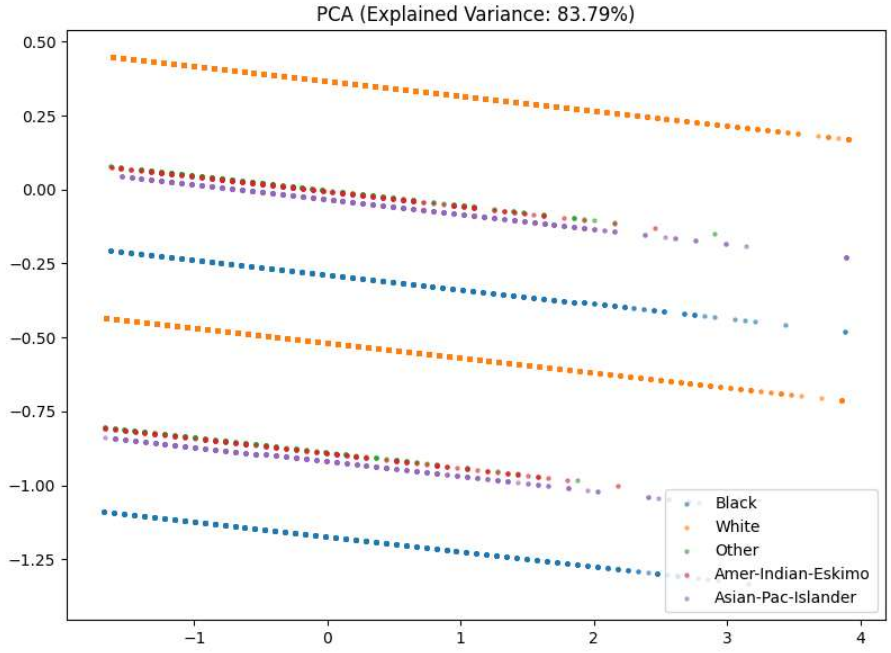
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X[:5000])
labels = df['race_label'].values[:5000]

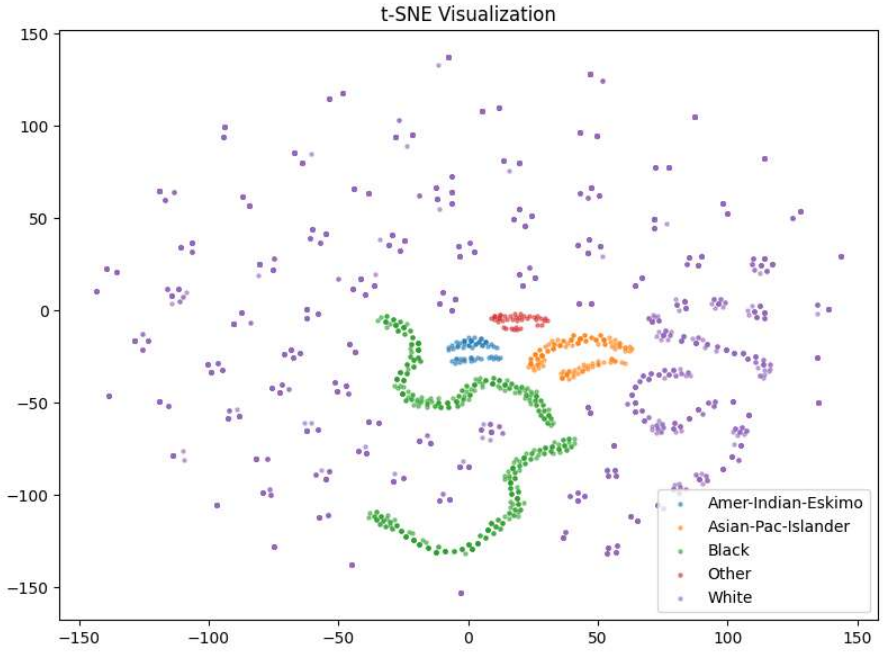
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for race in df['race_label'].unique():
    mask = df['race_label'] == race
    axes[0].scatter(X_pca[mask, 0], X_pca[mask, 1], label=race, alpha=0.5, s=5)
axes[0].set_title(f'PCA (Explained Variance: {pca.explained_variance_ratio_.sum():.2%})')
axes[0].legend()

for race in np.unique(labels):
    mask = labels == race
    axes[1].scatter(X_tsne[mask, 0], X_tsne[mask, 1], label=race, alpha=0.5, s=5)
axes[1].set_title('t-SNE Visualization')
axes[1].legend()

plt.tight_layout()
plt.show()
```






```
#data storytelling
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('adult_preprocessed.csv')

race_cols = [c for c in df.columns if c.startswith('race_')]
df['race_label'] = df[race_cols].idxmax(axis=1).str.replace('race_', '')

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

race_counts = df['race_label'].value_counts()
axes[0, 0].pie(race_counts.values, labels=race_counts.index, autopct='%1.1f%%', startangle=90)
axes[0, 0].set_title('STORY: The Dataset is Overwhelmingly White')

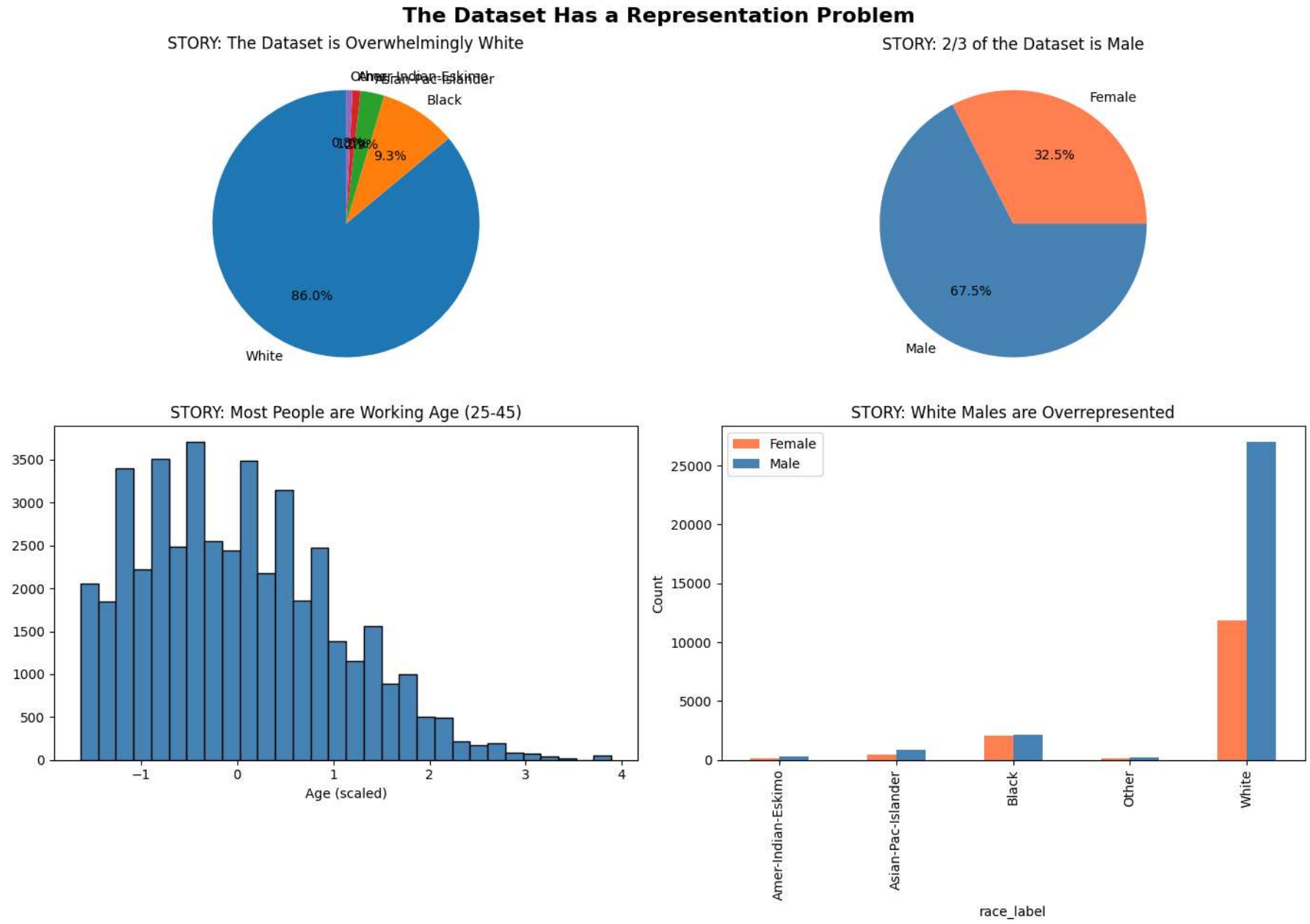
gender_counts = df['gender'].value_counts()
axes[0, 1].pie([gender_counts.get(0, 0), gender_counts.get(1, 0)],
               labels=['Female', 'Male'], autopct='%1.1f%%', colors=['coral', 'steelblue'])
axes[0, 1].set_title('STORY: 2/3 of the Dataset is Male')

axes[1, 0].hist(df['age'], bins=30, color='steelblue', edgecolor='black')
axes[1, 0].set_title('STORY: Most People are Working Age (25-45)')
axes[1, 0].set_xlabel('Age (scaled)')

pivot = pd.crosstab(df['race_label'], df['gender'])
pivot.plot(kind='bar', ax=axes[1, 1], color=['coral', 'steelblue'])
axes[1, 1].set_title('STORY: White Males are Overrepresented')
axes[1, 1].set_ylabel('Count')
```

```
axes[1, 1].legend(['Female', 'Male'])

plt.suptitle('The Dataset Has a Representation Problem', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
#Handling outliers in age and overrepresentation of white group (using weight formula: weight = total_records / (number_of_groups × count_of_this_group))
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Load and clean
df = pd.read_csv('adult.csv')
df.replace('?', np.nan, inplace=True)
df.dropna(inplace=True)
df = df[['age', 'race', 'gender', 'education', 'marital-status', 'occupation', 'relationship', 'native-country']]

# OUTLIER HANDLING (IQR Clipping)
scaler = StandardScaler()
df['age_scaled'] = scaler.fit_transform(df[['age']])

Q1 = df['age_scaled'].quantile(0.25)
Q3 = df['age_scaled'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df['age_cleaned'] = df['age_scaled'].clip(lower=lower_bound, upper=upper_bound)

print("=== OUTLIER HANDLING ===")
print(f"Before: min={df['age_scaled'].min():.3f}, max={df['age_scaled'].max():.3f}")
print(f"After:  min={df['age_cleaned'].min():.3f}, max={df['age_cleaned'].max():.3f}")
print(f"Outliers clipped: {len(df[df['age_scaled'] != df['age_cleaned']])}")

# ============================================================
# REPRESENTATION FIX (For ALL race and gender combinations)
# ============================================================

# Get unique values for race and gender
races = df['race'].unique()
genders = df['gender'].unique()
num_race_groups = len(races)
num_gender_groups = len(genders)
total = len(df)

# Calculate weight for each race (independent of gender)
race_weights = {}
for race in races:
    count = len(df[df['race'] == race])
    race_weights[race] = total / (num_race_groups * count)

# Calculate weight for each gender (independent of race)
gender_weights = {}
for gender in genders:
    count = len(df[df['gender'] == gender])
    gender_weights[gender] = total / (num_gender_groups * count)

# Apply weights
df['race_weight'] = df['race'].map(race_weights)
```

```
df['gender_weight'] = df['gender'].map(gender_weights)
df['sample_weight'] = df['race_weight'] * df['gender_weight']

print("\n=== RACE WEIGHTS ===")
for race, w in race_weights.items():
    print(f"{race}: {w:.4f}")

print("\n=== GENDER WEIGHTS ===")
for gender, w in gender_weights.items():
    print(f"{gender}: {w:.4f}")

print("\n=== COMBINED WEIGHTS FOR ALL GROUPS ===")
combined = df.groupby(['race', 'gender'])['sample_weight'].mean().reset_index()
combined.columns = ['Race', 'Gender', 'Sample Weight']
combined = combined.sort_values('Sample Weight', ascending=False)
print(combined.to_string(index=False))

# Save the new dataset
df.to_csv('adult_fixed.csv', index=False)
print("\nSaved to adult_fixed.csv")
```

```
=== OUTLIER HANDLING ===
Before: min=-1.630, max=3.893
After:  min=-1.630, max=2.796
Outliers clipped: 269

=== RACE WEIGHTS ===
Black: 2.1392
White: 0.2325
Other: 25.6215
Amer-Indian-Eskimo: 20.7917
Asian-Pac-Islander: 6.9412

=== GENDER WEIGHTS ===
Male: 0.7407
Female: 1.5387

=== COMBINED WEIGHTS FOR ALL GROUPS ===
               Race Gender  Sample Weight
              Other Female      39.423505
Amer-Indian-Eskimo Female      31.991948
              Other   Male      18.977574
Amer-Indian-Eskimo   Male      15.400192
Asian-Pac-Islander Female      10.680351
Asian-Pac-Islander   Male       5.141277
              Black Female       3.291508
              Black   Male       1.584457
              White Female       0.357723
              White   Male       0.172200

Saved to adult_fixed.csv
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('adult_fixed.csv')
```

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Panel 1: Original Race Distribution (Problem)
race_counts = df['race'].value_counts()
axes[0, 0].pie(race_counts.values, labels=race_counts.index, autopct='%1.1f%%', startangle=90)
axes[0, 0].set_title('PROBLEM: Dataset is Overwhelmingly White', fontsize=12, fontweight='bold')

# Panel 2: Original Gender Distribution (Problem)
gender_counts = df['gender'].value_counts()
axes[0, 1].pie([gender_counts.get('Female', 0), gender_counts.get('Male', 0)],
               labels=['Female', 'Male'], autopct='%1.1f%%', colors=['coral', 'steelblue'])
axes[0, 1].set_title('PROBLEM: 2/3 of Dataset is Male', fontsize=12, fontweight='bold')

# Panel 3: Age Distribution Before vs After Outlier Fix
axes[1, 0].hist(df['age_scaled'], bins=30, alpha=0.5, label='Original (with outliers)', color='red')
axes[1, 0].hist(df['age_cleaned'], bins=30, alpha=0.5, label='After IQR Clipping', color='green')
axes[1, 0].set_title('FIX: Outliers in Age Have Been Clipped', fontsize=12, fontweight='bold')
axes[1, 0].set_xlabel('Age (scaled)')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].legend()

# Panel 4: Sample Weights for ALL Race-Gender Combinations
combined = df.groupby(['race', 'gender'])['sample_weight'].mean().unstack()
combined.plot(kind='bar', ax=axes[1, 1], color=['coral', 'steelblue'])
axes[1, 1].set_title('FIX: Sample Weights for Every Race & Gender Group', fontsize=12, fontweight='bold')
axes[1, 1].set_ylabel('Mean Sample Weight')
axes[1, 1].set_xlabel('Race')
axes[1, 1].legend(['Female', 'Male'])
axes[1, 1].tick_params(axis='x', rotation=45)

plt.suptitle('Dataset Fixes: Outliers Clipped & Representation Weights Applied', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

